

Ch6. 합성곱 신경망2

6.1 이미지 분류를 위한 신경망

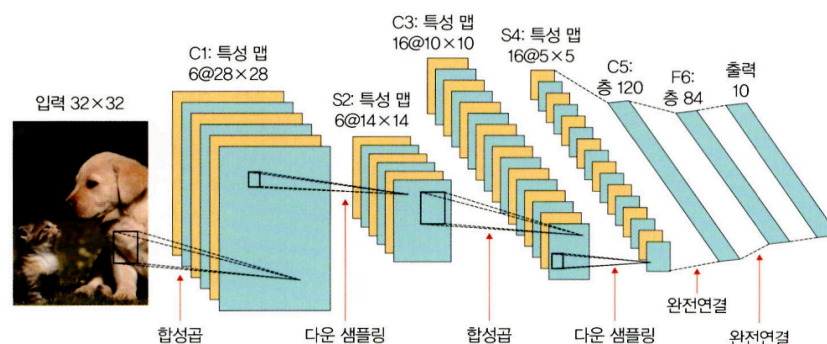
: 입력 데이터로 이미지를 사용한 분류는 특정 대상이 영상 내에 존재하는지 여부를 판단함.

▼ 1. LeNet-5

LeNet-5?

- LeNet-5 : 합성곱과 다운 샘플링(or 풀링)을 반복적으로 거쳐 마지막에 완전 연결층에서 분류를 수행하는 것
- 손글씨 숫자 인식하는 딥러닝 구조였는데 CNN의 초석이 되어버림

LeNet-5의 로직



- C1 : 5X5 합성곱 연산 → 28X28 크기의 특성 맵 6개 생성
- S2 : 다운 샘플링 → 14X14 크기의 특성 맵 6개 생성되도록 사이즈 줄임
- C3 : 5X5 합성곱 연산 → 10X10 크기의 특성 맵 16개 생성
- S4 : 다운 샘플링 → 5X5 크기의 특성 맵 16개 생성되도록 사이즈 줄임
- C5 : 5X5 합성곱 연산 → 1X1 크기의 특성 맵 120개 생성
- F6 : 완전연결층 → C5의 결과를 84개의 노드(유닛)에 연결

↳ C : 합성곱층(+활성화 함수) >> 커널 크기와 스트라이드 크기에 따라 풀링층으로 전달되는 특성 맵의 크기가 달라질수도?

↳ S : 풀링층 >> 스트라이드로 특성 맵의 크기 줄어든다

↳ F : 완전연결층

LeNet-5의 코드 구현

1. 라이브러리 호출

```
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms #이미지 변환 ( 전처리 ) 기능을 제공하는 라이브러리
from torch.autograd import Variable
from torch import optim #경사 하강법을 이용하여 가중치를 구하기 위한 옵티마이저 라이브러리
```

```

import torch.nn as nn
import torch.nn.functional as F
import os #파일 경로에 대한 함수들을 제공
import cv2
from PIL import Image
from tqdm import tqdm_notebook as tqdm #진행 상황을 가시적으로 표현해 주는데 , 특히 모델의 학습 경과를 확인하고 싶을 때 사용
import random
from matplotlib import pyplot as plt
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

```

2. 모델 학습에 필요한 데이터셋 전처리(텐서 변환)

```

# 이미지 데이터셋 전처리(텐서 변환)
class ImageTransforms:
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5,1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]),
            'val': transforms.Compose([
                transforms.Resize (256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }
    def __call__(self,img,phase):
        return self.data_transform[phase](img)

```

- `transforms.ToTensor()` : 이미지를 읽을 때 파이썬 라이브러리 PIL을 사용함 >> `ToTensor()` 함수로 배열을 `torch.FloatTensor` 배열로 바뀜 >> 이미지의 범위가 `[0.0,1.0]`(정규화)로, 배열의 차원 순서도 `C*H*W`로 바뀜
- **init** vs **call** : 인스턴스 초기화에 사용 vs 인스턴스 호출될 때 실행 & 리턴값 반환

3. 이미지 데이터셋 불러오기 > 훈련, 검증, 테스트로 분리

```

#이미지 데이터셋을 불러온 후 훈련, 검증, 테스트로 분리
cat_directory=r'/Users/bluecloud/Documents/대학/유런/data/chap06/dogs-vs-cats/Cat'
dog_directory=r'/Users/bluecloud/Documents/대학/유런/data/chap06/dogs-vs-cats/Dog/'

cat_images_filepaths=sorted([os.path.join(cat_directory, f) for f in
                                os.listdir(cat_directory)])
dog_images_filepaths=sorted([os.path.join(dog_directory, f) for f in
                                os.listdir(dog_directory)])
images_filepaths=[*cat_images_filepaths, *dog_images_filepaths] #개고양이 이미지 합쳐서 여기에 저장
correct_images_filepaths=[i for i in images_filepaths if cv2.imread(i) is not None]

random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths=correct_images_filepaths[:400] # 훈련용
val_images_filepaths=correct_images_filepaths[400:-10] # 검증용
test_images_filepaths=correct_images_filepaths[-10:] # 테스트용
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))

```

- sorted : 데이터를 정렬된 리스트로 만들어서 반환
- os.path.join : 디렉토리와 이미지 파일들을 합쳐 `/Users/bluecloud/Documents/대학/유연/data/chap06/dogs-vs-cats/Cat/cat_0.jpg` 이런식으로 표시해줌
- 이미지들을 랜덤으로 섞어주고 훈련용, 검증용, 테스트용으로 나눠줌

4. 이미지 데이터셋 클래스 정의

```
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'): # 경로만 지정
        self.file_list=file_list
        self.transform=transform
        self.phase=phase

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx): #데이터셋에서 데이터를 가져오는 부분으로 결과는 텐서 형태가 됨
        img_path=self.file_list[idx]
        img=Image.open(img_path) # 여기서 이미지 데이터 가져옴
        img_transformed=self.transform(img, self.phase) # 이미지에 train 전처리 적용
        label=img_path.split('/')[-1].split('.')[0] #이미지 데이터에 대한 레이블 값(dog, cat)을 가져옴
        if label=='dog':
            label=1
        elif label=='cat':
            label=0
        return img_transformed, label
```

- `label=img_path.split('/')[-1].split('.')[0]` : 이미지 데이터에 대한 레이블 값 가져오기 >> `/Users/bluecloud/Documents/대학/유연/data/chap06/dogs-vs-cats/Cat / cat . 0 . jpg` >> 여기에서 cat을 뽑는거 > / 를 기준으로 cat . 0 . jpg가 남고 . 을 기준으로 첫번째 값[0]인 cat을 뽑는다.

5. 이미지 데이터셋 클래스로 훈련 데이터와 검증 데이터 정의

```
train_dataset=DogvsCatDataset(train_images_filepaths, transform=ImageTransforms(size,
                                                                              mean, std), phase='train')
val_dataset=DogvsCatDataset(val_images_filepaths, transform=ImageTransforms(size,
                                                                              mean, std), phase='val')

index=0
print(train_dataset.__getitem__(index)[0].size()) # 훈련 데이터의 크기 출력
print(train_dataset.__getitem__(index)[1]) #훈련 데이터의 레이블
```

6. 데이터로더 정의

```
train_dataloader=DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_dataloader=DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict={'train': train_dataloader, 'val': val_dataloader} #훈련+검증해서 표현

batch_iterator=iter(train_dataloader)
inputs, label=next(batch_iterator)
print(inputs.size())
print(label)
```

- `train_dataloader=DataLoader(train_dataset, batch_size=batch_size, shuffle=True)` : 파라미터로 (1)데이터를 불러오기 위한 데이터셋, (2)배치 사이즈 (한 번에 가져오기는 무리니까), (3)임의로 섞어서 가져온다

7. 모델의 네트워크 클래스 생성 & 모델 생성

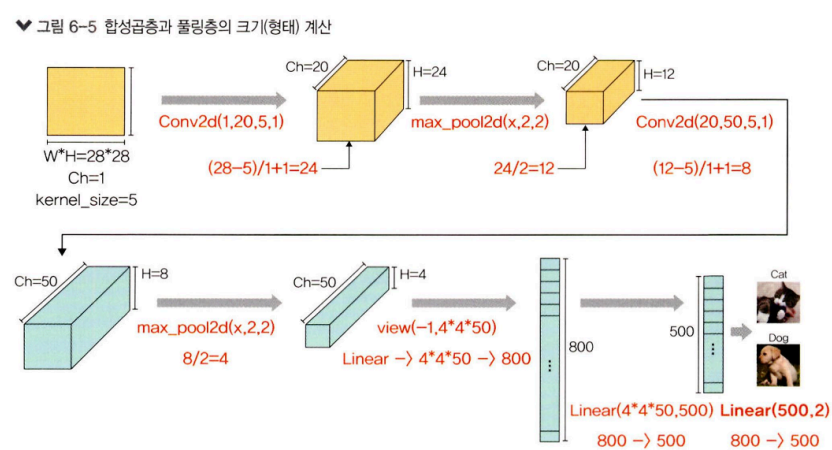
```
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.cnn1=nn.Conv2d(in_channels=3, out_channels=16, kernel_size=5, stride=1,
                             padding=0) #2d 합성곱층의 적용 : 입력은 아까 그거 torch.Size([3, 224, 224]) >> 출력은 (16,220,220)
        self.relu1=nn.ReLU()
        self.maxpool1=nn.MaxPool2d(kernel_size=2) # 최대 풀링 적용 >> 220/2되서 (16,110,110)
        self.cnn2=nn.Conv2d(in_channels=16, out_channels=32, kernel_size=5,
                             stride=1, padding=0) #2d 합성곱층의 적용 : 입력은 (16,110,110) >> 출력은 (32,106,106)
        self.relu2=nn.ReLU()
        self.maxpool2=nn.MaxPool2d(kernel_size=2) # 최대 풀링 적용 >> 106/2되서 (32,53,53)

        self.fc1=nn.Linear(32*53*53, 512)
        self.relu5=nn.ReLU()
        self.fc2=nn.Linear(512, 2)
        self.output=nn.Softmax(dim=1)

    def forward(self, x):
        out=self.cnn1(x)
        out=self.relu1(out)
        out=self.maxpool1(out)
        out=self.cnn2(out)
        out=self.relu2(out)
        out=self.maxpool2(out)
        out=out.view(out.size(0), -1) #완전 연결층에 데이터 전달하려면 1차원으로 형태 바뀌어야함
        out=self.fc1(out)
        out=self.relu5(out)
        out=self.fc2(out)
        out=self.output(out)
        return out

# 모델 생성
model=LeNet()
print(model)
```

- Conv2d 계층에서 출력 크기 구하는 공식 : $(\text{입력 데이터 크기 } W - \text{커널 크기 } F + 2 * \text{패딩 크기 } P) / \text{스트라이드 } S + 1$
- MaxPool2d 계층에서 출력 크기 구하는 공식 : $\text{입력 필터의 크기 } IF / \text{커널 크기 } F$
 - 여기서 입력 필터의 크기 IF는 앞의 Conv2d의 출력 크기이기도 함.



8. torchsummary로 모델 네트워크 구조 확인

- torchsummary 라이브러리 설치

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 16, 220, 220]	1,216
ReLU-2	[-1, 16, 220, 220]	0
MaxPool2d-3	[-1, 16, 110, 110]	0
Conv2d-4	[-1, 32, 106, 106]	12,832
ReLU-5	[-1, 32, 106, 106]	0
MaxPool2d-6	[-1, 32, 53, 53]	0
Linear-7	[-1, 512]	46,023,168
Linear-8	[-1, 2]	1,026
Softmax-9	[-1, 2]	0
Total params: 46,038,242		
Trainable params: 46,038,242		
Non-trainable params: 0		
Input size (MB): 0.57		
Forward/backward pass size (MB): 19.47		
Params size (MB): 175.62		
Estimated Total Size (MB): 195.67		

9. 옵티마이저와 손실 함수 정의 & 모델 파라미터와 손실 함수를 device(CPU)에 할당&모델 학습 함수 정의

```
#옵티마이저와 손실 함수 정의
optimizer=optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
criterion=nn.CrossEntropyLoss()
#모델의 파라미터와 손실 함수를 CPU에 할당
model=model.to(device)
criterion=criterion.to(device)

# 모델 학습 함수 정의 >> 학습 용도이므로 model.train() 사용
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):
    since=time.time()
    best_acc=0.0

    for epoch in range(num_epoch):
        print('Epoch {}/{}'.format(epoch+1, num_epoch))
        print('-'*20)

        for phase in ['train', 'val']:
            if phase=='train':
                model.train()
            else:
                model.eval()

            epoch_loss=0.0
            epoch_corrects=0

            for inputs, labels in tqdm(dataloader_dict[phase]): #dataloader_dict : 이거 훈련 데이터셋이 될듯
                inputs=inputs.to(device)
                labels=labels.to(device)
                optimizer.zero_grad()

                with torch.set_grad_enabled(phase=='train'):
                    outputs=model(inputs)
                    _, preds=torch.max(outputs, 1)
                    loss=criterion(outputs, labels)

                if phase=='train':
                    loss.backward() # 모든 파라미터에 대해 기울기 계산
                    optimizer.step() # 파라미터 갱신

            epoch_loss+=loss.item()*inputs.size(0)
            epoch_corrects+=torch.sum(preds==labels.data)

        epoch_loss=epoch_loss/len(dataloader_dict[phase].dataset)
```

```

epoch_acc=epoch_corrects.double()/len(dataloader_dict[phase].dataset)
print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

if phase=='val' and epoch_acc>best_acc: # 검증 데이터셋에 대해 가장 최적의 정확도 저장
    best_acc=epoch_acc
    best_model_wts=model.state_dict()

time_elapsed=time.time()-since
print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed//60, time_elapsed%60))
print('Best val Acc: {:.4f}'.format(best_acc))
return model

```

- `epoch_loss+=loss.item()*inputs.size(0)` : 손실 함수의 reduction 파라미터가 `criterion=nn.CrossEntropyLoss()`이므로 기본값인 `reduction='mean'`이므로 전체 오차를 배치 크기로 나눠 반환하므로 전체 오차만 구하기 위해 오차에 입력 크기를 곱해준다.

10. 모델 학습

```

# 모델 학습 >> train_model 출력
import time

num_epoch=10
model=train_model(model, dataloader_dict, criterion, optimizer, num_epoch)

```

```

val Loss: 0.6949 Acc: 0.5435
Training complete in 6m 43s
Best val Acc: 0.597826

```

11. 모델 테스트를 위한 함수 정의

```

#정확도 측정해서 csv 파일로 저장
import pandas as pd

id_list=[]
pred_list=[]
_id=0
with torch.no_grad(): # 역전파 중 텐서에 대한 변화도 계산할 필요가 없음을 의미 >> 훈련 데이터셋 학습과 가장 큰 차이점
    for test_path in tqdm(test_images_filepaths):
        img=Image.open(test_path)
        _id=test_path.split('/')[-1].split('.')[1]
        transform=ImageTransforms(size, mean, std)
        img=transform(img, phase='val')
        img=img.unsqueeze(0) #텐서에 차원을 추가
        img=img.to(device)

        model.eval()
        outputs=model(img)
        preds=F.softmax(outputs, dim=1)[:, 1].tolist() #지정된 차원을 따라 텐서의 요소가 (0,1) 범위에 있고, 합계가 1이 되도록 조정
        id_list.append(_id)
        pred_list.append(preds[0])
res=pd.DataFrame({
    'id': id_list,
    'label': pred_list
})

res.sort_values(by='id', inplace=True)

```

```
res.reset_index(drop=True, inplace=True)
res.to_csv('/Users/bluecloud/Documents/대학/유런/data/chap06/predictions.csv', index=False)
```

- `unsqueeze(0)` : 텐서에 차원을 추가할 때 사용 >> 0 위치니까 여기서는 (1,x,y) : 채널 추가시켜서 3차원
- `F.softmax(outputs, dim=1)` : outputs에 차원(dim=1)을 따라 텐서의 요소가 합계가 1이 되도록 조정

	id	label
0	109	0.564653
1	145	0.451740
2	15	0.609144
3	162	0.531125
4	167	0.561944
5	200	0.507271
6	210	0.661168
7	211	0.680880
8	213	0.465212
9	224	0.656852

- 예측 결과 : 0.5보다 크면 개, 작으면 고양이

12. 예측 결과 이미지 출력(테스트 데이터셋을 파라미터로 전달)

```
# 테스트 데이터셋 이미지를 출력하기 위한 함수 >> 0.5보다 크면 개, 0.5보다 작으면 고양이
class_=classes={0: 'cat', 1:'dog'} # 개묘에 대한 클래스 정의
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows=len(images_filepaths)//cols
    figure, ax=plt.subplots(nrows=rows, ncols=cols, figsize=(12,6))
    for i, image_filepath in enumerate(images_filepaths):
        image=cv2.imread(image_filepath)
        image=cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        a=random.choice(res['id'].values)
        label=res.loc[res['id']==a, 'label'].values[0]
        if label>0.5:
            label=1
        else:
            label=0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()
# 예측 결과 이미지 출력(테스트 데이터셋을 파라미터로 전달)
display_image_grid(test_images_filepaths)
```



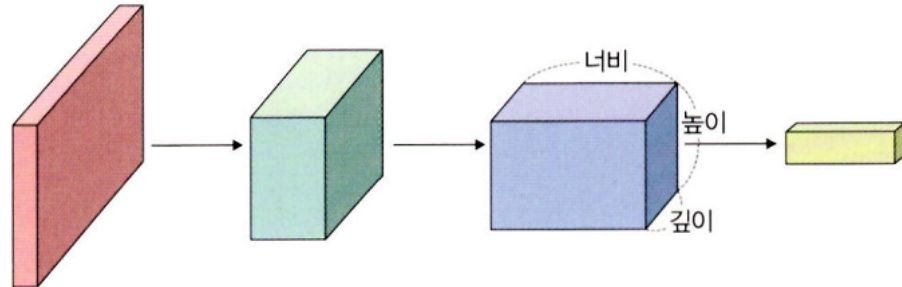
- 음...부르

▼ 2. AlexNet

AlexNet?

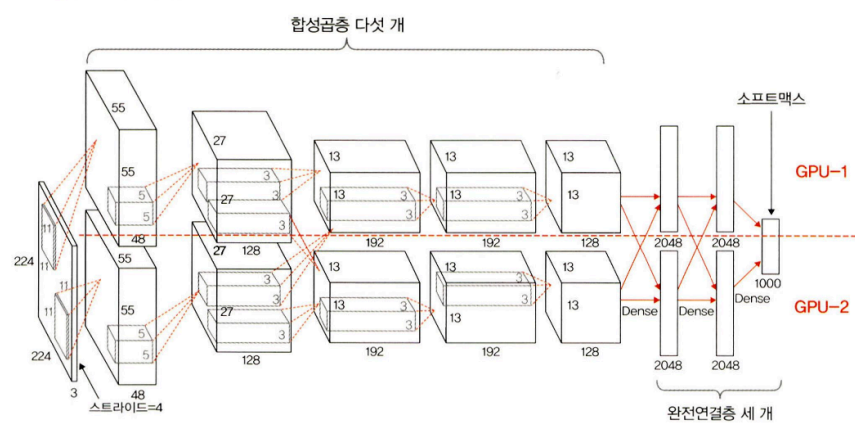
- ImageNet 영상 데이터베이스를 기반으로 한 대회에서 우승한 CNN 구조
- CNN에서 그림은 3차원 구조를 갖고 합성곱을 거치면서 특성맵이 만들어지고 이에 따라 중간 영상의 깊이가 달라짐

▼ 그림 6-8 CNN 구조



- AlexNet의 구조 : 합성곱층(ReLU) 5개 + 완전연결층 3개(카테고리 1000개 분류 >> softmax함수 활용)
- 병렬 구조인 것 빼고는 LeNet-5랑 비슷

▼ 그림 6-9 AlexNet 구조



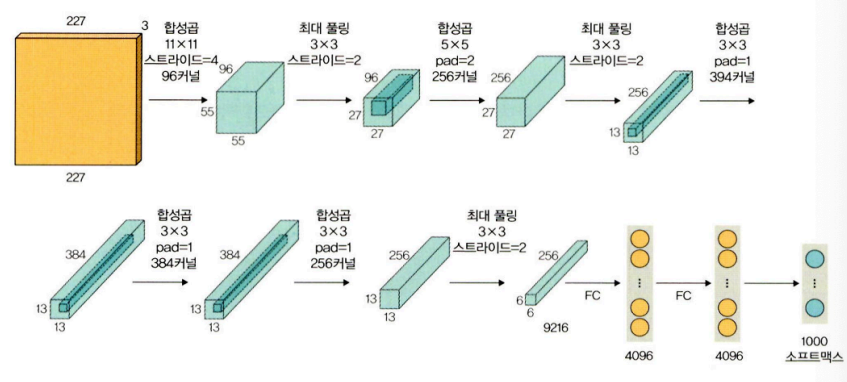
계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	227×227	—	—	—
합성곱층	96	55×55	11×11	4	렐루(ReLU)
최대 풀링층	96	27×27	3×3	2	—
합성곱층	256	27×27	5×5	1	렐루(ReLU)
최대 풀링층	256	13×13	3×3	2	—
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	256	13×13	3×3	1	렐루(ReLU)
최대 풀링층	256	6×6	3×3	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

- 네트워크에 학습 가능한 변수는 총 6600만개
- 입력은 227*227*3크기의 RGB벡터 / 각 클래스에 해당하는 1000*1 확률 벡터를 출력

AlexNet의 코드 구현

아래의 예제에서는 마지막에 클래스 두 개만 사용함

▼ 그림 6-11 AlexNet 예제 네트워크



1. 데이터 전처리 > 데이터 불러와서 훈련, 검증, 테스트 용도로 분리 > 커스텀 데이터셋 정의 > 변수(mean, std 등) 값 정의 > 훈련, 검증, 테스트 데이터셋 정의 > 데이터 로더로 전달하여 cpu로 불러올 준비
2. AlexNet 모델 네트워크 정의 및 모델 객체 생성

AlexNet 모델 네트워크 정의 : 합성곱 Conv2d + 활성화 함수 ReLu + 풀링층 MaxPool2d >> 5번 반복 & 두개의 완전 연결층과 출

```
class AlexNet(nn.Module):
    def __init__(self) → None:
        super(AlexNet, self).__init__()
        self.features=nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 192, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(192, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
        )
        self.avgpool=nn.AdaptiveAvgPool2d((6,6)) #풀링을 위해 사용
        self.classifier=nn.Sequential(
            nn.Dropout(),
            nn.Linear(256*6*6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 512),
            nn.ReLU(inplace=True),
            nn.Linear(512, 2),
        )

    def forward(self, x: torch.Tensor) → torch.Tensor:
        x=self.features(x)
        x=self.avgpool(x)
        x=torch.flatten(x,1)
        x=self.classifier(x)
        return x

# 모델 객체 생성
model=AlexNet()
model.to(device)
```

```
AlexNet(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU(inplace=True)
    (10): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU(inplace=True)
    (12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(6, 6))
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=9216, out_features=4096, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=4096, out_features=512, bias=True)
    (5): ReLU(inplace=True)
    (6): Linear(in_features=512, out_features=2, bias=True)
  )
)
```

- ReLU(inplace=True) : inplace가 T인 이유는 기존 값을 연산 결과값으로 대체하여 기존의 값을 무시한다는 의미이다.
- AdaptiveAvgPool2d((6,6)) : 풀링을 위해 사용 >> 풀링 작업이 끝날 때 필요한 출력 크기를 정의

3. AlexNet 모델 네트워크 정의 >> LeNet이랑 같은 코드

```
#옵티마이저 및 손실 함수 정의
optimizer=optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
criterion=nn.CrossEntropyLoss()

# 모델의 네트워크 시각화 >> torchsummary
from torchsummary import summary
summary(model, input_size=(3,256,256))

def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):

    since=time.time()
    best_acc=0.0

    for epoch in range(num_epoch):
        print('Epoch {}/{}'.format(epoch+1, num_epoch))
        print('-'*20)

        for phase in ['train', 'val']:
            if phase=='train':
                model.train()
            else:
                model.eval()

            epoch_loss=0.0
            epoch_corrects=0

            for inputs, labels in tqdm(dataloader_dict[phase]):
                inputs=inputs.to(device)
                labels=labels.to(device)
                optimizer.zero_grad()

                with torch.set_grad_enabled(phase=='train'):
                    outputs=model(inputs)
                    _, preds=torch.max(outputs, 1)
                    loss=criterion(outputs, labels)

                if phase=='train':
                    loss.backward()
                    optimizer.step()

            epoch_loss+=loss.item()*inputs.size(0)
            epoch_corrects+=torch.sum(preds==labels.data)
```



```

        epoch_loss=epoch_loss/len(dataloader_dict[phase].dataset)
        epoch_acc=epoch_corrects.double()/len(dataloader_dict[phase].dataset)
        print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

    time_elapsed=time.time() -since
    print('Training complete in {:.0f}m {:.0f}s'.format(
        time_elapsed//60, time_elapsed%60))
    return model

```

4. 모델 학습하고 예측하기

```

#모델 학습
num_epoch=10
model=train_model(model, dataloader_dict, criterion, optimizer, num_epoch)

# 모델을 이용한 예측 >> 테스트 데이터셋 예측
import pandas as pd
id_list=[]
pred_list=[]
_id=0
with torch.no_grad():
    for test_path in tqdm(test_images_filepaths):
        img=Image.open(test_path)
        _id=test_path.split('/')[-1].split('.')[1]
        transform=ImageTransforms(size, mean, std)
        img=transform(img, phase='val')
        img=img.unsqueeze(0)
        img=img.to(device)

        model.eval()
        outputs=model(img)
        preds=F.softmax(outputs, dim=1)[:, 1].tolist()

        id_list.append(_id)
        pred_list.append(preds[0])

res=pd.DataFrame({
    'id': id_list,
    'label': pred_list
})

res.to_csv('/Users/bluecloud/Documents/대학/유런/data/chap06/AlexPredict.csv', index=False)

```

5. 데이터 프레임과 시각적으로 표현하는 함수로 결과 확인

```

#데이터 프레임 결과 확인
res.head(10)

#예측 결과를 시각적으로 표현하기 위한 함수
class_=classes={0: 'cat', 1:'dog'}
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows=len(images_filepaths)//cols
    figure, ax=plt.subplots(nrows=rows, ncols=cols, figsize=(12,6))
    for i, image_filepath in enumerate(images_filepaths):

```

```

image=cv2.imread(image_filepath)
image=cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

a=random.choice(res['id'].values)
label=res.loc[res['id']==a, 'label'].values[0]
if label>0.5:
    label=1
else:
    label=0
ax.ravel()[i].imshow(image)
ax.ravel()[i].set_title(class_[label])
ax.ravel()[i].set_axis_off()
plt.tight_layout()
plt.show()

#예측 결과에 대해 이미지와 함께 출력
display_image_grid(test_images_filepaths)

```

	id	label
0	145	0.496658
1	211	0.497313
2	162	0.496273
3	200	0.498622
4	210	0.497938
5	224	0.496485
6	213	0.496858
7	109	0.497442
8	15	0.497316
9	167	0.496192



- 그렇게 잘진 않음 >> 데이터셋이 많으면 성능 좋아질수도..?

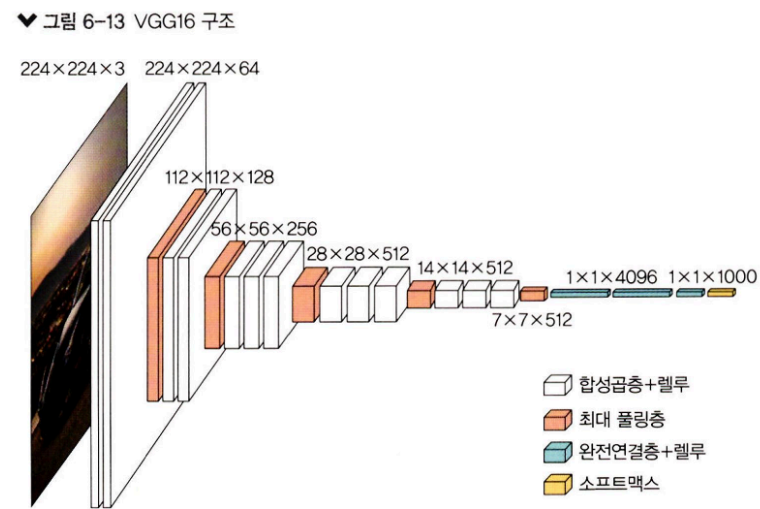
▼ 3. VGGNet

VGGNet이란?

- VGGNet : 합성곱층의 파라미터 수를 줄이고 훈련 시간을 개선하려고 탄생한 것 >> 네트워크의 깊이에 따른 성능을 확인하고자 네트워크 계층의 개수에 따른 여러 유형들이 있음 VGG16, VGG19등..
- 네트워크 깊이의 영향을 확인하는게 최대 목표 → 합성곱층에 사용하는 커널의 크기를 3X3으로 고정

VGG16

- 파라미터가 총 1억 3300만개, 합성곱 커널의 크기는 3X3, 최대 풀링 커널의 크기는 2X2, 스트라이드는 2 ⇒ 결과적으로 64개의 224X224 특성맵이 생성
- 마지막 16계층 계층을 제외하고는 ReLU 활성화 함수가 적용됨.



- VGGn은 어떻게 구현함? 덜 쌓거나 더 깊게 쌓아올리면 됨.

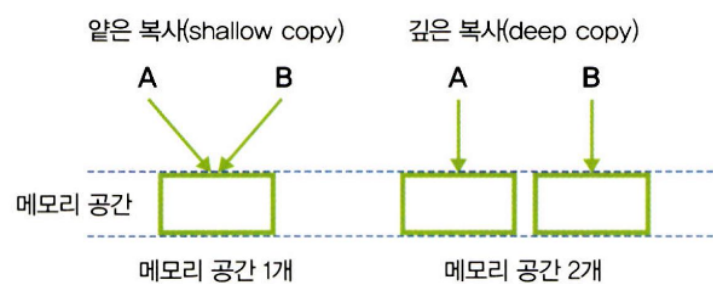
VGGNet의 코드 구현

1. 라이브러리 호출

```
import copy #객체 복사를 위해 사용
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as Datasets

device=torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

- 객체 복사 `import copy`
 - 얕은 복사 : `copy.copy()` → 메모리 공간 1개를 share하는거, 한 쪽에서 변경되면 복사된 다른쪽도 변경됨
 - 깊은 복사 : `copy.deepcopy()` → 아예 새로운 메모리 공간을 새로 만들어주는거 → 다른 객체



2. VGG 모델 정의

```
class VGG(nn.Module):
    def __init__(self, features, output_dim):
        super().__init__()
        self.features=features #VGG 모델에 대한 매개변수에서 받아온 features 값을 self.features여기에 넣어줌
        self.avgpool=nn.AdaptiveAvgPool2d(7)
        self.classifier=nn.Sequential(
            nn.Linear(512*7*7, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(4096, 4096),
```

```

nn.ReLU(inplace=True),
nn.Dropout(0.5),
nn.Linear(4096, output_dim)
) # 완전 연결층과 출력층 정의

```

```

def forward(self, x):
    x=self.features(x)
    x=self.avgpool(x)
    h=x.view(x.shape[0], -1)
    x=self.classifier(h)
    return x, h

```

3. 모델 유형의 정의 >> VGG11, VGG13, VGG16, VGG19

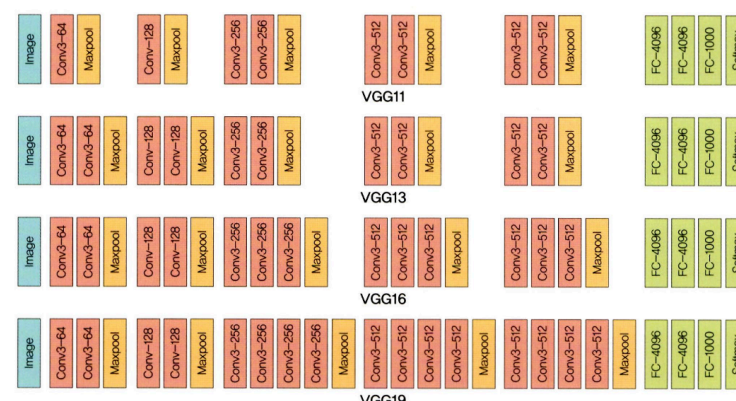
```
vgg11_config=[64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
```

```
vgg13_config=[64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
```

```
vgg16_config=[64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M',
              512, 512, 512, 'M']
```

```
vgg19_config=[64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512,
              512, 'M', 512, 512, 512, 512, 'M']
```

- 여기서 11,13,16,19는 전체 계층의 개수 / 풀링층의 개수는 3, 나머지 차가 합성곱층의 개수
- 숫자 : 출력 채널로 Conv2d로 수행하라는 의미이며, 다음 계층에서 입력 채널이 된다.
- M : 최대풀링을 수행하라



4. VGG 계층 정의 → 계층 생성(배치 정규화에 대한 계층 추가)

```

def get_vgg_layers(config, batch_norm):
    layers=[]
    in_channels=3

    for c in config: #vgg11_config값들을 가져오기 >> 위에서 정의된 숫자와 문자의 값들이 실질적으로 기능으로 구현됨
        assert c=='M' or isinstance(c, int)
        if c=='M': # M이면 최대 풀링 적용
            layers+= [nn.MaxPool2d(kernel_size=2)]
        else: # 숫자이면 합성곱 Conv2d 적용
            conv2d=nn.Conv2d(in_channels, c, kernel_size=3, padding=1)
            if batch_norm: # 배치 정규화면 배치 정규화 + ReLU 적용
                layers+= [conv2d, nn.BatchNorm2d(c), nn.ReLU(inplace=True)]
            else: #아니면 렐루만
                layers+= [conv2d, nn.ReLU(inplace=True)]
            in_channels=c

```

```
return nn.Sequential(*layers)
```

#모델 계층 생성 >> 배치 정규화에 대한 계층 추가

```
vgg11_layers=get_vgg_layers(vgg11_config, batch_norm=True)
```

#VGG11 계층 확인

```
print(vgg11_layers)
```

```
Sequential(
  (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): ReLU(inplace=True)
  (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (6): ReLU(inplace=True)
  (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (10): ReLU(inplace=True)
  (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (13): ReLU(inplace=True)
  (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (17): ReLU(inplace=True)
  (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (20): ReLU(inplace=True)
  (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (23): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (24): ReLU(inplace=True)
  (25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (26): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (27): ReLU(inplace=True)
  (28): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)
```

5. VGG11 사전 훈련된 모델 사용

```
import torchvision.models as models
```

```
pretrained_model=models.vgg11_bn(pretrained=True) #vgg11_bn: VGG11 기본 모델에 배치 정규화가 적용된 모델 사용
```

```
print(pretrained_model)
```

- 사실 이미 훈련된 VGG11가 있어서 튜닝도 완료된 상태임 >> 바로 가져다쓰기
- `vgg11_bn` : VGG11 기본 모델에 배치 정규화 적용된 모델 사용하기
- `pretrained=True` : 사전 훈련된 모델 사용

6. 이미지 데이터 전처리 → ImageFolder를 이용하여 데이터셋 불러오기 → 훈련과 검증 데이터 분할(9:1) → 검증 데이터 전처리 → 메모리로 데이터 불러오기(배치 크기만큼 나누어서 가져오기)

7. 옵티마이저와 손실 함수 정의 및 모델 정확도 측정 함수

#옵티마이저와 손실 함수 정의

```
optimizer=optim.Adam(model.parameters(), lr=1e-7)
```

```
criterion=nn.CrossEntropyLoss()
```

```
model=model.to(device)
```

```
criterion=criterion.to(device)
```

#모델 정확도 측정 함수

```
def calculate_accuracy(y_pred, y):
```

```
    top_pred=y_pred.argmax(1, keepdim=True)
```

```
    correct=top_pred.eq(y.view_as(top_pred)).sum()
```

```
    acc=correct.float()/y.shape[0]
```

```
    return acc
```

- `correct=top_pred.eq(y.view_as(top_pred)).sum()` : 예측과 정답이 일치하는 경우 그 개수를 합쳐서 correct에 저장 >> `y.view_as(top_pred)` : y에 대한 텐서 크기를 `top_pred`에 대한 텐서 크기로 변경

8. 모델 학습 함수 정의, 모델 성능 측정 함수, 학습 시간 측정 함수

```
#모델 학습 함수 정의 >> 훈련 데이터셋을 이용
def train(model, iterator, optimizer, criterion, device):
    epoch_loss=0
    epoch_acc=0

    model.train()
    for (x,y) in iterator:
        x=x.to(device)
        y=y.to(device)

        optimizer.zero_grad()
        y_pred, _=model(x)
        loss=criterion(y_pred, y)
        acc=calculate_accuracy(y_pred, y)
        loss.backward()
        optimizer.step()

        epoch_loss+=loss.item()
        epoch_acc+=acc.item()
    return epoch_loss/len(iterator), epoch_acc/len(iterator)

#모델 성능 측정 함수 >> 검증 및 테스트 데이터셋을 이용
def evaluate(model, iterator, criterion, device):
    epoch_loss=0
    epoch_acc=0

    model.eval()
    with torch.no_grad():
        for(x,y) in iterator:
            x=x.to(device)
            y=y.to(device)
            y_pred, _=model(x)
            loss=criterion(y_pred, y)
            acc=calculate_accuracy(y_pred, y)
            epoch_loss+=loss.item()
            epoch_acc+=acc.item()
    return epoch_loss/len(iterator), epoch_acc/len(iterator)

#학습 시간 측정 함수 >> 모델의 학습 시간(시작, 종료)를 측정하기 위한 함수
def epoch_time(start_time, end_time):
    elapsed_time=end_time-start_time
    elapsed_mins=int(elapsed_time/60)
    elapsed_secs=int(elapsed_time-(elapsed_mins*60))
    return elapsed_mins, elapsed_secs
```

9. 모델 학습

```
EPOCHS = 5
best_valid_loss = float('inf')
for epoch in range(EPOCHS) :
    start_time = time.monotonic()
    train_loss, train_acc = train(model, train_iterator, optimizer, criterion, device)
    valid_loss, valid_acc = evaluate(model,valid_iterator, criterion, device)
```



```

if valid_loss < best_valid_loss: #valid_loss가 가장 작은 값을 구해 그 상태의 모델을 VGG-model.pt여기에 저장
    best_valid_loss = valid_loss
    torch.save(model.state_dict(),'/Users/bluecloud/Documents/대학/유런/data/chap06/VGG-model.pt')
end_time = time. monotonic()
epoch_mins, epoch_secs = epoch_time(start_time, end_time) # 모델 훈련에 대한 시작과 종료 시간을 저장

print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
print(f'\tTrain Loss: {train_loss:.3f} | Train Acc: {train_acc*100: .2f}%')
print(f'\t Valid. Loss: {valid_loss:.3f}|Valid. Acc: {valid_acc*100:.2f}%')

```

```

Epoch: 01 | Epoch Time: 5m 18s
Train Loss: 0.700 | Train Acc: 50.70%
Valid. Loss: 0.693|Valid. Acc: 43.40%
Epoch: 02 | Epoch Time: 5m 38s
Train Loss: 0.700 | Train Acc: 48.10%
Valid. Loss: 0.693|Valid. Acc: 43.40%
Epoch: 03 | Epoch Time: 6m 35s
Train Loss: 0.707 | Train Acc: 48.52%
Valid. Loss: 0.693|Valid. Acc: 43.40%
Epoch: 04 | Epoch Time: 5m 49s
Train Loss: 0.699 | Train Acc: 48.79%
Valid. Loss: 0.693|Valid. Acc: 41.51%
Epoch: 05 | Epoch Time: 6m 8s
Train Loss: 0.696 | Train Acc: 53.17%
Valid. Loss: 0.693|Valid. Acc: 58.49%

```

10. VGG-model.pt에 저장된 모델의 테스트 데이터셋에 대한 성능 측정

```

#VGG-model.pt을 불러와 테스트 데이터셋에 대한 성능 측정
model.load_state_dict(torch.load('/Users/bluecloud/Documents/대학/유런/data/chap06/VGG-model.pt'))
test_loss, test_acc=evaluate(model, test_iterator, criterion, device)
print(f'Test Loss: {test_loss:.3f} | Test Acc: {test_acc*100:.2f}%')

```

```

Test Loss: 0.692 | Test Acc: 58.33%

```

11. 예측 결과 이미지 출력

```

# get_predictions()의 반환값으로 모델의 가장 정확한 예측값을 추출
images, labels, probs=get_predictions(model, test_iterator)
pred_labels=torch.argmax(probs,1)
corrects=torch.eq(labels, pred_labels)
correct_examples=[]

for image, label, prob, correct in zip(images, labels, probs, corrects):
    if correct:
        correct_examples.append((image, label, prob))

correct_examples.sort(reverse=True, key=lambda x: torch.max(x[2], dim=0).values)

# 이미지 출력을 위한 전처리
def normalize_image(image):
    image_min=image.min()
    image_max=image.max()
    image.clamp_(min=image_min, max=image_max)
    image.add_(-image_min).div_(image_max-image_min+1e-5)
    return image

#모델이 정확하게 예측한 이미지 출력 함수
import matplotlib.pyplot as plt

def plot_most_correct(correct, classes, n_images, normalize=True):
    rows=int(np.sqrt(n_images))
    cols=int(np.sqrt(n_images))

```

```

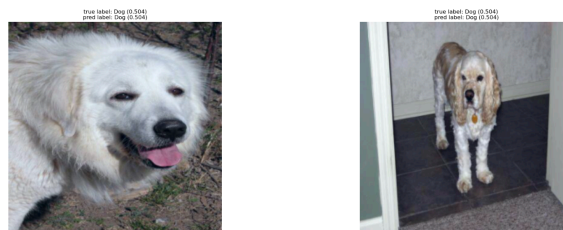
fig=plt.figure(figsize=(25,20))
for i in range(rows*cols):
    ax=fig.add_subplot(rows, cols, i+1)
    image, true_label, probs=correct[i]
    image=image.permute(1,2,0) #축 변경
    true_prob=probs[true_label]
    correct_prob, correct_label=torch.max(probs, dim=0)
    true_class=classes[true_label]
    correct_class=classes[correct_label]

    if normalize:
        image=normalize_image(image)

    ax.imshow(image.cpu().numpy())
    ax.set_title(f'true label: {true_class} ({true_prob:.3f})\n\
                f'pred label: {correct_class} ({correct_prob:.3f})')
    ax.axis('off')
fig.subplots_adjust(hspace=0.4)

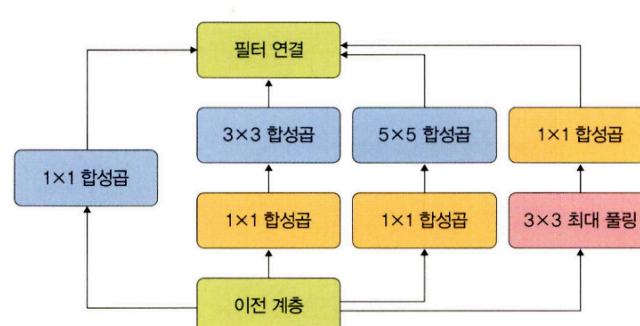
# 예측 결과 이미지 출력
classes=test_dataset.classes
N_IMAGES=5
plot_most_correct(correct_examples, classes, N_IMAGES)

```



▼ 4. GoogLeNet

- GoogLeNet이란? H/W자원을 최대한 효율적으로 이용하면서 학습 능력을 극대화시키는 깊고 넓은 신경망
- 깊고 넓은 신경망은 어떻게? 인셉션 모듈의 추가로



↳ 1X1, 3X3, 5X5 합성곱 연산을 각각 수행 for 특징을 효율적으로 추출

↳ 3X3 : 최대 풀링은 입력과 출력의 높이와 너비가 같아야 하므로 '드물게' 패딩을 추가

- 희소 연결이 적용된 GoogLeNet
 - 관련성 높은 노드끼리만 연결하는 방법 <> CNN(밀집)
 - 연산량이 적어져 과적합 해결 가능
- GoogLeNet은 어디에? 딥러닝 대회에서 대용량 데이터로 학습을 진행할 때 심층 신경망의 과적합이나 기울기 소멸 문제, 시간 지연 및 연산 속도를 해결해줌.

